[MLOps][FDD] Playback-less Rosbag → T4Dataset Conversion Pipeline Design

This document proposes a rosbag → t4dataset pipeline design. t4dataset → annotation format is out of scope.

@Yihsiang FANG @Max SCHMELLER Take a look and let's discuss

Input

Required pipeline inputs:

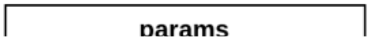
- pilot-auto repository and hash
  - e.g. `git@github.com:tier4/pilot-auto.x2.git` `v4.4.0`
- `vehicle_model` : e.g. `j6_gen2`
- `sensor_model` : e.g. `aip_x2_gen2`
- `vehicle_id` : e.g. `j6_gen2_16`
- `rosbag_ids` : one or more MOB rosbag_ids
- `start_time` , `end_time` : double
- `topic_information` : largely preset but necessary for different vehicles
- *Other parameters can be considered but these are the required ones*

Build Process

- Github repository and commit for autoware (pilot_auto.x2 `dev/offline`)
  - Create image: `jazzy-x2-dev-offline` and make this resuable
- Short-term
  - Base image: `ros:jazzy` Build necessary packages via colcon + offline processing pipeline
  - Optimization: CI for small build
- Long-term: move key packages to build farm, use pixi

Conversion Flow

Draw.IO



Pseudo-code

```
# Params
rosbag_paths: List[str]          # ordered bag files
lidar_calibrations: dict         # per-lidar calibration files
tf_static: dict                  # sensor frame → base_link transforms
converter_params: ConverterParams

# === Indexing Pass ===
tf_buffer, topic_metadata = rosbag_indexing(rosbag_paths)

# === Module Initialization ===
decoders        = NebulaDecoder(lidar_calibration, converter_param) # for each lidar
correctors      = DistortionCorrection(tf_static, converter_params) # for each lidar
concatenator    = Concatenator(tf_static, converter_params) # one
t4_writer       = T4DatasetWriter(converter_params)
output_bag      = RosbagWriter(converter_params)

# === Main Loop ===
for topic, msg, stamp in iterate_messages(rosbag_paths):
    # Output rosbag
    if topic in passthrough_topics:
        output_bag.write_raw(topic, msg, stamp)

    # Packet topics
    if topic in lidar_packet_topics:
        cloud = decoders.process_packet(topic, msg)
        if cloud:
            undist_status, undistorted = correctors.undistort(topic, cloud)
            concat_status, concatenated = concatenator.process_cloud(topic, undistorted)
            if status == "complete" or \
               (status == "timeout" and converter_params.allow_frame_drop)
                concat_pc2, lidar_info = concatenated
                concat_stamp = stamp
                required_images_ts = self.sync_images_ts(topic_metadata)
                t4_writer.add_sample(result, ego_pose, image_buffers)
                output_bag.write(result)

    # IMU, Twist Topic, maybe GNSS
    elif topic in imu_twist_gnss_topics:
        imu_twist_gnss_buffers.update(topic, stamp, msg)
        correctors.enqueue(topic, msg)

    # Images
    elif topic in image_topics:
        image_buffers.update(topic, stamp, msg)

    # Dataset ready
    if concat_pc2 and required_images_ts in image_buffers:
        images = image_buffers[required_images_ts]
        ego_pose = tf_buffer.lookup_ego_pose(concat_stamp)
        imu, twist, gnss = imu_twist_gnss_buffers[concat_stamp]
        radar, can_msgs = ...
        t4_writer.write_frame(concat, images, imu, twist, gnss, ego_pose)

# === Finalize ===
t4_writer.finalize()
output_bag.close()
```

Refactoring Concerns

- Concatenate issues we probably need to solve:
  - `AdvancedMatchingStrategy` class needs to be able to take offset/noise vectors directly instead of using `rclcpp::Node`
  - `CloudCollector` timer needs to be removed and probably changed to some actual `is_timed_out(double sensor_time)` function which can trigger `concatenate_callback()` if timed out
  - `cloud_callback()` `get_clock()->now()` probably replaced with msg stamp
  - `CombineCloudHandler` tf2 static_transforms should accept pre-loaded eigen matrices instead of live tf2 buffer
  - current call chain is `process_pointcloud(topic, cloud)` → `concatenate_callback()` → `ros2_parent_node_->publish_clouds(ConcatenatedCloudResult&&)` which doesn't expose actual results
    - probably we need some `on_complete_(result, info)` called in `concatenate_callback()` which can give status and `std::optional<ConcatenatedResult>`